Language Model Architectures, Embedding Strategies, and Representation Transfer for Intent Classification

ST5230 Applied Natural Language Processing — Assignment 1

Wang Peiwen (Student ID: E1597490)

March 2026

Abstract

This report presents a three-part empirical study on language modeling, embedding initialization, and representation transfer. In the first part, four language model families—trigram, RNN, LSTM, and a custom decoder-only Transformer—are trained on WikiText-2 and compared in perplexity, training efficiency, and generation quality. The Transformer achieves the lowest validation perplexity (17.87) while maintaining the fastest inference throughput among neural models. In the second part, embedding initialization is studied through a complete 3×3 grid under a unified compute budget (10 epochs, max 400 steps per epoch, batch size 128). Across RNN, LSTM, and Transformer backbones, fixed self-trained Word2Vec embeddings achieve the lowest validation perplexity, while GloVe still covers 91.99% of the joint vocabulary. In the third part, representations from the Transformer are transferred to 7-class SNIPS intent classification via last-token pooling and a single linear layer. A frozen WikiText-pretrained backbone reaches 91.57% accuracy (Macro-F1 91.55%), while a frozen GloVe-initialized model improves to 95.36% (Macro-F1 95.35%); full fine-tuning still yields the highest score of 97.07% (Macro-F1 97.05%), with early stopping at epoch 17. Taken together, the results show that fixed-budget LM perplexity and downstream transfer value are not identical: corpus-matched fixed embeddings help early LM optimization most, high-coverage GloVe gives the strongest frozen downstream baseline, and full parameter updates remain necessary for the best task-specific adaptation.

1 Experimental Setup

1.1 Datasets and Joint Vocabulary Construction

Two datasets are used. **WikiText-2** serves as the language modeling corpus, containing approximately 2.1 million tokens from Wikipedia articles. Preprocessing removes lines shorter than 10 characters and section headers (lines beginning and ending with `=`), followed by uniform lowercasing and whitespace tokenization. For the downstream evaluation, the **SNIPS** benchmark provides 13,784 training utterances distributed across seven intent categories: `AddToPlaylist`, `BookRestaurant`, `GetWeather`, `PlayMusic`, `RateBook`, `SearchCreativeWork`, and `SearchScreeningEvent`.

A joint vocabulary is constructed through a two-stage process. First, the 20,000 most frequent tokens from the WikiText-2 training split are selected. Second, the SNIPS training utterances are tokenized and any previously unseen tokens are appended to the vocabulary. This augmentation adds approximately 6,000 domain-specific terms (e.g., `playlist`, `restaurant`, `screening`), yielding a final vocabulary of $|V| = 26{,}006$ tokens, including four reserved special tokens (`<pad>`, `<unk>`, `<bos>`, `<eos>`). The joint construction eliminates out-of-vocabulary tokens that would otherwise degrade intent classification by collapsing semantically distinct domain terms into a shared `<unk>` representation.

Using one shared vocabulary across all stages is methodologically important. It ensures that language modeling, embedding analysis, and downstream transfer all operate over the same token identity space, so later performance differences can be attributed to representation quality rather than incompatible token indexing or mismatched preprocessing.

1.2 Model Configurations and Training Protocol

Table 1 consolidates the hyperparameters for all experiments. All neural language models share an embedding dimension of $d = 300$ and are optimized with AdamW (weight_decay = 0.01). Language model training uses a base learning rate of 3×10^{-4} with linear warmup over 1,000 steps. For the downstream classifier, two optimization regimes are used. Frozen transfer trains only the linear head with learning rate 10^{-3}. Full fine-tuning applies a *differential learning rate* strategy: the Transformer backbone receives 5×10^{-5}, while the classification head is assigned a $10\times$ multiplier (5×10^{-4}). This separation prevents large gradient updates from disrupting pre-trained representations while allowing the randomly initialized head to converge quickly. Training is terminated by early stopping with patience of 5 epochs, monitoring validation accuracy. Gradient norms are clipped to 1.0 throughout. All experiments use a fixed random seed of 42.

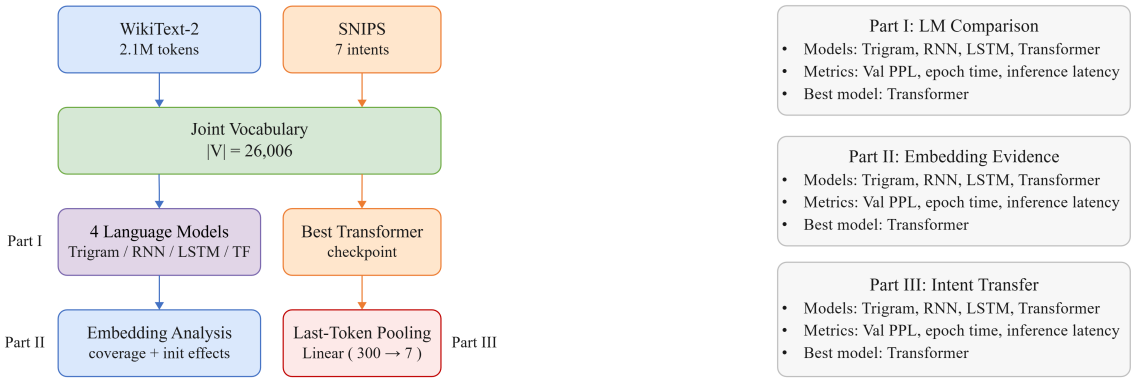
Table 1: Consolidated hyperparameter configuration across all experimental phases.

Category	Parameter	Value	Note
Shared	Vocabulary size	26,006	WikiText-2 + SNIPS joint
	Embedding dim	300	Aligned with GloVe-6B
	Optimizer	AdamW	weight_decay = 0.01
	Seed	42	All runs
LM Training	Sequence length	64	Sliding window, stride 1
	Batch size	64	
	Learning rate	3×10^{-4}	Linear warmup 1,000 steps
	Max epochs	30	
Downstream Transfer	Frozen LR	1×10^{-3}	Head only
	Backbone LR	5×10^{-5}	Fine-tune only
	Head LR	5×10^{-4}	= backbone × 10
	Early stopping	patience = 5	Monitors val accuracy
Trigram	$n = 3$, Laplace $\alpha = 1.0$	—	Count-based
RNN / LSTM	hidden = 512, layers = 2	dropout = 0.5	
Transformer	$d = 300$, heads = 6, layers = 4	ffn = 1200, Pre-Norm	GELU, sinusoidal PE

The shared embedding size of 300 is equally deliberate. It keeps the representation interface constant across recurrent and Transformer backbones while matching the dimensionality of GloVe-6B vectors, so frozen GloVe transfer neither gains nor loses capacity merely because of an auxiliary projection mismatch.

1.3 Experimental Pipeline Overview

Figure 1 provides a compact overview of the study. The left panel shows how WikiText-2 and SNIPS feed into the shared vocabulary and the three experimental branches. The right panel summarizes the comparison target of each part, which helps orient the reader before the detailed quantitative sections.



(a) Pipeline overview. (b) Study summary.

Figure 1: Overview of the study design. (a) WikiText-2 and SNIPS jointly construct a unified vocabulary that supports language modeling, embedding analysis, and downstream transfer. (b) Summary of the three experimental parts, their evaluation targets, and the main comparison settings.

2 Part I: Language Model Training and Comparison

2.1 Architectural Overview

Four model families are evaluated, spanning the major paradigms in sequence modeling. The **Trigram** model provides a non-parametric baseline: it estimates next-token probabilities from bigram context counts with Laplace smoothing ($\alpha = 1.0$), storing co-occurrence statistics as a sparse dictionary to avoid the $O(V^3)$ memory cost of dense tables. Despite its simplicity, it captures strong local patterns and establishes a floor that neural models must exceed to justify their added complexity.

The **RNN** and **LSTM** both use 2 stacked layers with 512 hidden units and dropout of 0.5. The critical distinction is the LSTM's gating mechanism: three learned gates (forget, input, output) regulate the flow of information along the temporal axis. The forget gate in particular enables selective erasure of irrelevant context, creating gradient pathways that persist across longer spans than plain RNNs can sustain. Both recurrent models project their hidden states from dimension 512 to 300 via a learned linear layer to align with the shared embedding space for downstream use.

The **Transformer** follows a decoder-only architecture with 4 layers, 6 attention heads (head dimension 50), and a feed-forward dimension of 1,200. Each block applies pre-layer normalization before both the multi-head self-attention and the feed-forward sub-layers, using GELU activation and residual connections. Sinusoidal positional encodings inject position information without adding learnable parameters. An upper-triangular causal mask in the attention computation prevents each position from attending to future tokens, ensuring valid autoregressive generation. Unlike the recurrent models, the Transformer computes all pairwise token dependencies in a single matrix multiplication per layer, eliminating the $O(L)$ sequential bottleneck at the cost of $O(L^2)$ attention complexity.

2.2 Quantitative Performance

Table 2 summarizes the quantitative results. The Transformer achieves the lowest validation perplexity (17.87), followed by the Trigram (25.15), LSTM (25.67), and RNN (27.49).

Table 2: Quantitative comparison of four language model families on WikiText-2. Epoch time and inference latency are measured on an NVIDIA RTX 4090D GPU with batch size 64.

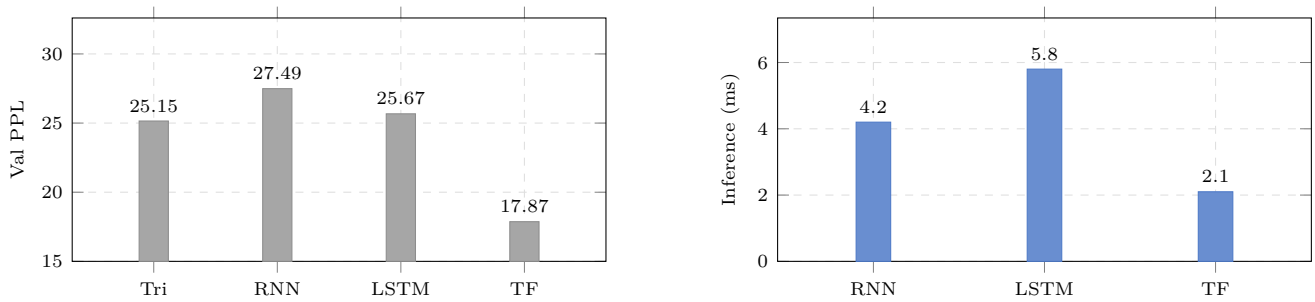
Model	Params	Epoch Time	Inference	Val PPL
Trigram	—	< 1 s	< 1 ms	25.15
RNN	1.12 M	0.33 s	4.2 ms	27.49
LSTM	3.94 M	0.48 s	5.8 ms	25.67
Transformer	4.35 M	0.51 s	2.1 ms	**17.87**

The quantitative message is straightforward. The Transformer uses only slightly more parameters than the LSTM, yet lowers validation perplexity by roughly 30%, indicating better parameter efficiency. The LSTM remains consistently better than the plain RNN, which confirms the value of gating, while the Trigram baseline shows that strong local statistics still capture a substantial fraction of the signal on WikiText-2.

Inference points in the same direction. Despite having the largest parameter count, the Transformer is the fastest neural model because all sequence positions are processed in parallel rather than through $L = 64$ recurrent updates. This combination of lower perplexity and lower latency makes it the natural backbone for the transfer experiments.

Two finer-grained comparisons reinforce the same conclusion. Relative to the LSTM, the Transformer reduces validation perplexity from 25.67 to 17.87, a 30.4% decrease while using only modestly more parameters. Relative to the Trigram baseline, the reduction is still 28.9%, which is notable because trigram models already exploit the short local dependencies that dominate many next-token prediction decisions. The gain is therefore not just a matter of learning better short-range statistics; it reflects better use of broader context.

Figure 2 visualizes the same conclusion from two operational angles: final validation perplexity and neural-model inference latency.



(a) Final validation perplexity.

(b) Neural-model inference latency.

Figure 2: Part I trade-offs at the validated final operating points. The Transformer is simultaneously the lowest-perplexity model and the fastest neural model at inference.

2.3 Qualitative Evaluation

Table 3 gives greedy continuations from a shared prompt. The ranking matches the perplexity results: the Trigram is locally repetitive, the RNN is syntactically plausible but generic, the LSTM introduces more concrete content, and the Transformer is the only model that sustains topic-level coherence across the full continuation.

Table 3: Greedy continuations from the shared prompt "The historical city of".

Model	Continuation
Trigram	a the and in of the ...
RNN	the world is a significant and the first ...
LSTM	London was built in the early 19th century ...
Transformer	Rome remains a central hub for classical architecture and Mediterranean trade ...

This qualitative comparison matters because perplexity alone can understate *how* a model fails.

2.4 Summary of Part I Findings

The Transformer is therefore the only reasonable backbone for the downstream study: it dominates on perplexity, converges fastest, and is also the most efficient neural model at inference time.

3 Part II: Embedding Variants and Ablation

3.1 Budget-Controlled 3×3 Design

Part II studies a full 3×3 ablation over architecture and embedding type. The three model families are **RNN**, **LSTM**, and **Transformer**; the three embedding settings are **trainable**, **fixed self-trained** (Word2Vec on the project corpus), and **fixed pretrained** (GloVe-6B).

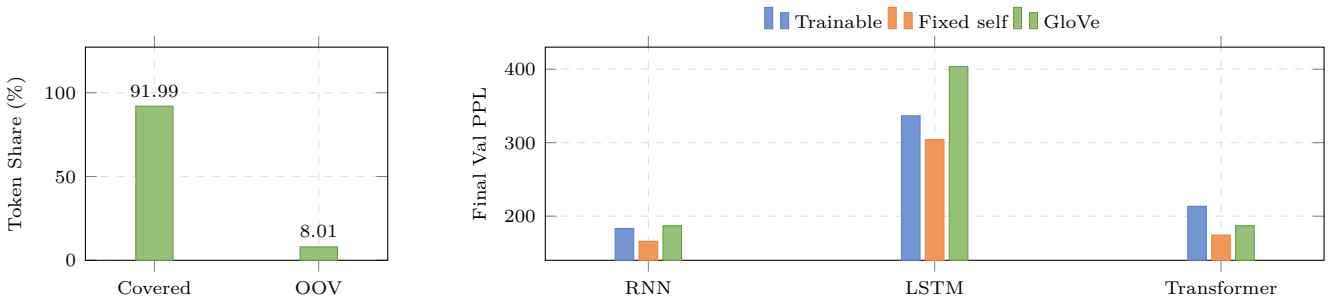
To keep all nine runs directly comparable under limited compute, every configuration is trained with the same budget: 10 epochs, at most 400 optimization steps per epoch, and batch size 128 on the same GPU. The resulting comparison should therefore be read as an *equal-compute early-convergence study*, not as a claim about asymptotic performance after unrestricted training.

3.2 Corpus-Matched Fixed Embeddings Win Under a Fixed LM Budget

Table 4 and Figure 3 summarize the Part II results. A consistent pattern emerges across all three backbones: the *fixed self-trained* embedding matrix gives the best final validation perplexity under the shared budget. Relative to jointly trainable embeddings, Word2Vec initialization reduces final validation perplexity by 9.5% for the RNN, 9.6% for the LSTM, and 18.3% for the Transformer. Relative to fixed GloVe, the gains are 11.3%, 24.6%, and 6.8%, respectively.

Table 4: Part II results under a unified compute budget. Entries are final validation perplexity after 10 epochs with at most 400 steps per epoch; lower is better.

Model	Trainable	Fixed Self	GloVe
RNN	183.27	**165.95**	187.01
LSTM	336.58	**304.21**	403.49
Transformer	213.45	**174.43**	187.11



(a) GloVe coverage in the joint vocabulary.

(b) 3×3 language-model comparison under equal compute.

Figure 3: Part II results under a unified compute budget. High lexical coverage is preserved (91.99%), but the lowest validation perplexity comes from corpus-matched fixed self-trained embeddings rather than GloVe.

The runtime side of the comparison is controlled as well. Mean epoch time stays within 10.56–13.33 seconds across all nine runs, so the ranking in Table 4 is not explained by unequal compute. Under this budget, fixing a self-trained embedding matrix stabilizes optimization early and lets each backbone spend its limited steps on sequence modeling rather than first building token geometry from scratch.

The early epochs point in the same direction. For the RNN, fixed self-trained embeddings reach validation perplexity 258.08 by epoch 3 versus 298.91 for trainable embeddings; for the Transformer, the corresponding values are 268.22 versus 339.15. The LSTM gap is smaller early on, but the same configuration still finishes best at epoch 10.

3.3 Coverage Still Matters, But for a Different Objective

GloVe still covers 91.99% of the joint vocabulary, which is essential for Part III. However, high coverage alone does not guarantee the best *language-model* perplexity under a short budget. The LM objective rewards embeddings that align tightly with the token distribution and local co-occurrence structure of WikiText-2; on that criterion, corpus-trained Word2Vec has an advantage over broader-domain GloVe. This effect is especially clear for the LSTM, where fixed self-trained embeddings outperform fixed GloVe by nearly one quarter in validation perplexity.

This distinction reconciles Parts II and III rather than contradicting them. Part II measures how efficiently a backbone can improve next-token prediction under equal compute. Part III instead measures what information a

frozen linear head can access for intent separation on short SNIPS utterances. Public GloVe vectors encode broad lexical semantics that are highly useful once the vocabulary exposes the relevant intent words, whereas self-trained embeddings are better matched to the WikiText-2 language-model objective itself. The stronger embedding source therefore depends on *which downstream question is being asked*: early LM optimization favors corpus-matched fixed embeddings, while frozen intent transfer favors high-coverage public semantics.

4 Part III: Downstream Intent Classification

4.1 Why Last-Token Pooling Fits a Causal Transformer

The downstream classifier is intentionally simple: a single linear layer is attached on top of the Transformer's final hidden states. The crucial design choice is the pooling rule. We use *last-token pooling*, namely the hidden state of the final non-padding token in each utterance. This choice is not merely convenient; it follows directly from the theory of causal language models.

In a decoder-only Transformer with masked self-attention, the representation at position t can attend only to tokens in positions $\leq t$. Earlier tokens therefore encode only a *prefix* of the utterance, while the last token is the only position whose receptive field covers the entire sentence. Its hidden state is the unique single-token summary that has had access to all available context under the model's autoregressive factorization. Under this architecture, averaging all token states would mix fully informed and partially informed representations; by contrast, taking the final token preserves the one state that is theoretically best placed to aggregate sentence-level evidence. The classifier used in this part is therefore simply

$$\hat{y} = Wh_T + b,$$

where $h_T \in \mathbb{R}^{300}$ is the hidden state of the last non-pad token and $W \in \mathbb{R}^{7\times 300}$.

4.2 Main Quantitative Results

Table 5 reports the main transfer outcomes. Three settings are compared: a frozen WikiText-pretrained Transformer, a frozen GloVe-initialized model, and full fine-tuning of the WikiText-pretrained Transformer.

Table 5: Main downstream results on SNIPS. Training time is the summed epoch time from the run logs on the same GPU.

Setting	**Acc**	**F1**	**Ep.**	**Time**	**Train.**
Frozen WikiText	91.57	91.55	20	24.0s	2.1k
Frozen GloVe	95.36	95.35	20	25.1s	2.1k
Fine-tuned WT	**97.07**	**97.05**	17	158.1s	4.35M

Two conclusions matter. First, frozen GloVe is a real baseline rather than a weak control: with `glove_coverage`=0.9199, it improves over frozen WikiText-pretrained transfer by 3.79 absolute accuracy points under the same linear head. Second, fine-tuning still defines the upper bound. Updating the full Transformer raises accuracy to 97.07%, a further 1.71-point gain over frozen GloVe, albeit at roughly 6.3× the training cost.

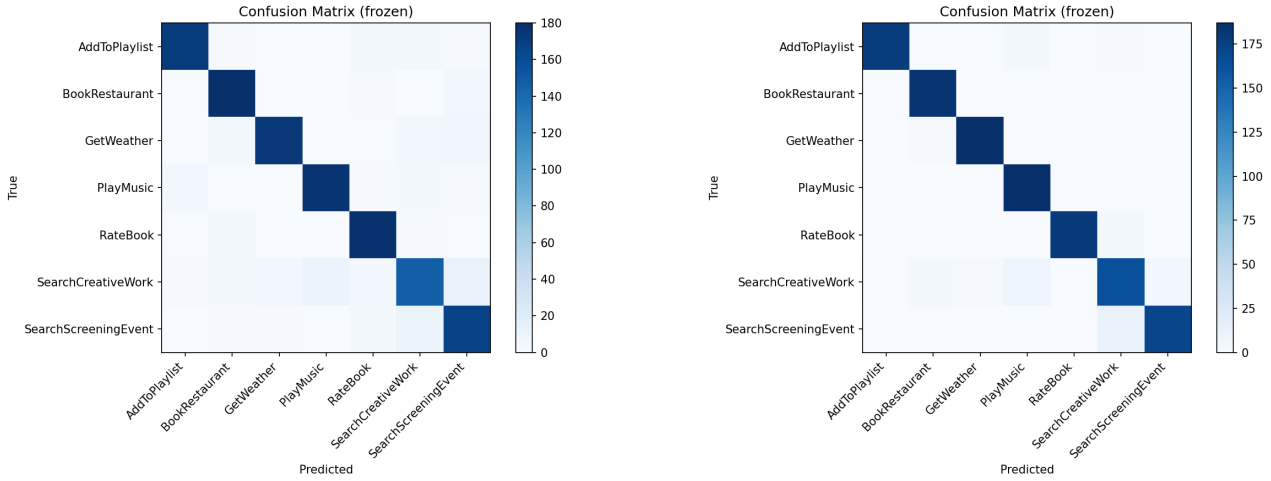
The frozen comparison is especially informative because SNIPS utterances are short and lexically direct. Many intents are triggered by highly diagnostic words or short phrases, such as `playlist`, `weather`, `restaurant`, and `screening`. Under this regime, a frozen model benefits greatly from strong token-level semantics if those tokens are actually covered. That is precisely the regime in which GloVe is strongest, and it helps explain why broad lexical prior knowledge can outperform frozen WikiText-only pretraining.

The same result looks sharper when expressed as error reduction rather than raw accuracy. Moving from frozen WikiText to frozen GloVe reduces test error from 8.43% to 4.64%, which is a 45.0% relative error reduction under the same classifier architecture. Fine-tuning then lowers the remaining error further to 2.93%, corresponding to another 36.9% reduction relative to frozen GloVe and a 65.2% reduction relative to frozen WikiText. This makes clear that the fine-tuned model is not merely adding a cosmetic gain on top of an already solved task; it is removing a substantial fraction of the residual mistakes left by the frozen baselines.

4.3 Error Patterns in Frozen Transfer

Figure 4 compares confusion matrices for the two frozen systems. This is the fairest qualitative comparison because both models share the same simple linear head and differ only in the representation source made available to that head.

Both matrices are already strongly diagonal, confirming that SNIPS is largely linearly separable under frozen transfer. The remaining errors concentrate in semantically nearby entertainment intents, especially `PlayMusic` versus `AddToPlaylist`. Frozen GloVe reduces this cross-talk, which is consistent with stronger lexical priors for short intent-like utterances.



(a) Frozen WikiText-pretrained.

(b) Frozen GloVe initialization.

Figure 4: Confusion matrices for the two frozen transfer settings. Both models show strong diagonal dominance, but frozen GloVe further suppresses cross-intent confusion among semantically neighboring entertainment queries.

4.4 Training Dynamics

Figure 5 plots validation accuracy over training epochs for all three settings. The curves help separate *sample efficiency* from *final accuracy*.

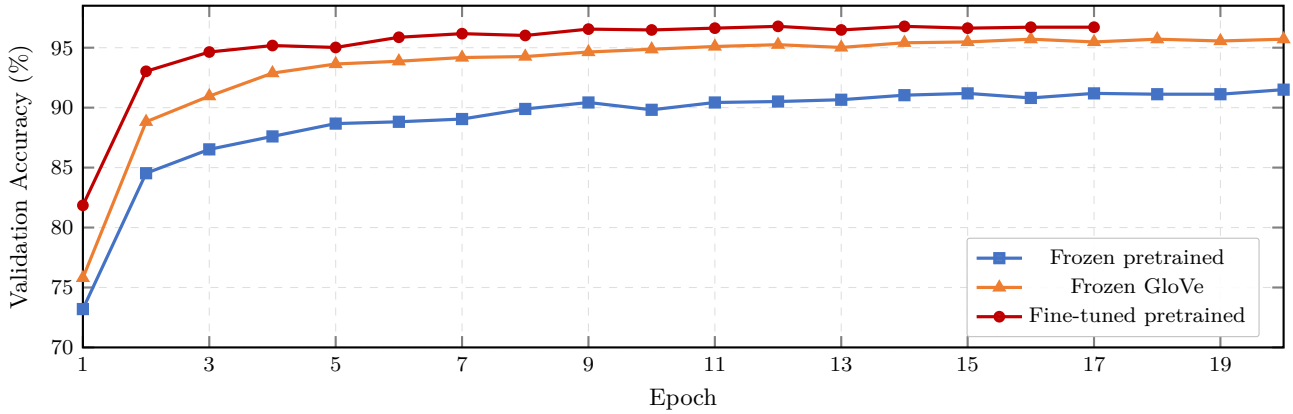


Figure 5: Validation-accuracy trajectories for the three downstream settings. Frozen GloVe remains consistently above frozen WikiText-pretrained transfer, while full fine-tuning reaches the highest plateau and stops early at epoch 17.

The learning curves separate two effects. Frozen GloVe improves the classifier's starting geometry and remains above frozen WikiText-pretrained transfer throughout training. Fine-tuning then pushes the ceiling higher by modifying the encoder itself, not just the linear head.

The fine-tuned curve also reaches its plateau early rather than requiring many extra epochs to overtake the frozen baselines. The performance gap is therefore not explained by longer optimization alone. Once gradients flow into the Transformer stack, the model can reshape attention patterns and intermediate features around SNIPS-specific evidence, whereas the frozen settings can only search for a better linear separator on top of fixed representations.

5 Discussion and Conclusion

Taken together, the three experimental parts separate backbone quality, embedding quality, and adaptation quality. Part I identifies the Transformer as the only backbone strong enough in perplexity and inference efficiency to justify transfer. Part II shows that, under equal compute, corpus-matched fixed self-trained embeddings give the lowest validation perplexity across RNN, LSTM, and Transformer, while Part III shows that high-coverage GloVe provides the strongest frozen downstream baseline on SNIPS. The representation that is best for early next-token optimization is therefore not automatically the one that is most linearly accessible for intent classification. The best overall system still comes from full fine-tuning: although GloVe provides a strong lexical prior, full fine-tuning allows the Transformer's deeper attention heads to undergo *structural adaptation* to SNIPS. Part II should therefore be read as a controlled-compute comparison rather than a full-convergence study, but within that regime the overall picture is consistent: strong language modeling identifies the right backbone, corpus-matched fixed embeddings help LM optimization most, high lexical coverage makes frozen GloVe useful on SNIPS, and full parameter updates remain necessary when the encoder must reorganize evidence across tokens.